

ZeroRun: Reproducible test-result reuse for AI coding tools

Jishan Kapoor*

Independent researcher, Toronto, Canada

Abstract

ZeroRun is open-source software for auditable validation handoffs between AI coding tools. A consumer can inspect whether a deterministic test command previously succeeded under the same declared source and runtime identity, or request fresh execution for diagnostics. The package combines authenticated status records, isolated execution, command-line and Model Context Protocol interfaces, and reproducible evaluation tools. Eligible hits avoid source staging. The four-target study, completed through an explicit post-crash recovery, reconciles 168 fresh comparisons and 96 reused successes. Separate fresh evaluation verified 1 completed agent fix among 2 selected real-issue pilot cases. The evaluation separates correctness checks, controlled hand-off costs, and model-selected use. Reused status does not replay historical output or establish patch correctness beyond the requested tests. ZeroRun supports investigation of when validation reuse is useful; universal workflow acceleration is not established.

Keywords: AI coding tools, test-result reuse, research software, reproducibility, software testing

*Corresponding author.

Email address: `kapoorjishan2@gmail.com` (Jishan Kapoor)

Required Metadata

Current code version

Nr.	Code metadata description	Value
C1	Current code version	ZeroRun 0.5.2; research release 2026-09-07; historical study runtime 0.5.1 retained separately
C2	Permanent GitHub link to code/repository used for this code version	<code>floxy-21/zerorun-research</code> ; linked immutable source/evidence snapshot
C3	Legal Code License	MIT (ZeroRun code); third-party licenses retained; trajectory data CC-BY-4.0
C4	Code versioning system used	Git
C5	Software code languages, tools, and services used	Python; pytest; Docker; CLI; Model Context Protocol (MCP); Codex skill
C6	Compilation requirements, operating environments & dependencies	Runtime: Python ≥ 3.10 ; no core Python dependencies. Whole-task reuse: Linux/amd64 and Docker. Canonical analysis: CPython 3.12–3.14. Experiments: digest-pinned Python 3.12 container and frozen dependencies.
C7	If available Link to developer documentation/manual	Version-pinned README: installation, interfaces, reproduction, and limitations
C8	Support email for questions	<code>kapoorjishan2@gmail.com</code>

Table 1: Code metadata. The public release preserves the tested runtime source and provides a separate, documented research harness.

1. Motivation and significance

Repository-level AI coding tools use test results when deciding whether an edit is satisfactory. SWE-bench and SWE-agent make this interaction a concrete research setting [1, 2]. A producer may validate a patch before handing the workspace to another tool or reviewer. The consumer then needs to distinguish an identified earlier success from newly executed evidence: the same shell command can observe different source, dependencies, or external state.

ZeroRun exposes the previous successful exit status of a deterministic test command under an unchanged declared execution identity. Its users are testing-tool integrators and researchers evaluating validation handoffs. A consumer that only needs the earlier status can request reuse; one needing a new transcript executes fresh. This makes the benefit and cost of each choice measurable. ZeroRun neither learns a caching policy nor predicts patch correctness.

Regression-test selection avoids tests considered unaffected by a change [3, 4]. Existing build and LLM-tool caches already address result reuse (Table 2); eidos provides adjacent LLM function-integration software [5]. LabChain combines hash-based intermediate-result reuse with modular scientific pipelines and JSON configuration [6]. ZeroRun instead exposes an earlier test-status decision under an operator-qualified execution identity. Neither content hashing nor attaching a cache to an AI interface is a new algorithm here.

The contribution is an inspectable validation handoff for operator-qualified, result-only test tasks: a repository-bound status interface, a hit path that avoids source staging, and an executable evaluation kit linking declared input boundaries, independent fresh outcomes, and full request cost.

2. Software description

2.1. Architecture and execution contract

Figure 1 shows the package’s manifest validation, input fingerprinting, result storage, isolated execution, and interfaces. A version-2 whole-task manifest identifies a command, declared input paths, environment policy, and digest-pinned Linux/amd64 container. Its supported cache contract requires a result-only computation, no output artifacts, and no cached stdout or stderr. The input closure and determinism are operator assumptions, not facts inferred from passing tests.

The identity includes declared content, relevant command-source content, directory modes, command and environment policy, and inspected runtime

System	Reuse basis	Returned information / execution state
Bazel [7]	Build-action hashes with declared inputs and outputs	Action metadata, output files, stdout and stderr.
ToolCaching [8]	Tool name and serialized arguments; feature/TTL-based admission	Tool results and metadata; requests classified as COMMAND excluded.
TVCache [9]	Tool-call prefixes; optional state-preserving annotations	Tool values; selective sandbox snapshots support continued rollout execution.
ZeroRun	Reviewed local test inputs, command and runtime identity	Identified earlier success; no historical transcript or restored output artifacts.

Table 2: Operating contracts, not a performance ranking. ZeroRun evaluates a narrower result-only task contract; no head-to-head superiority is established.

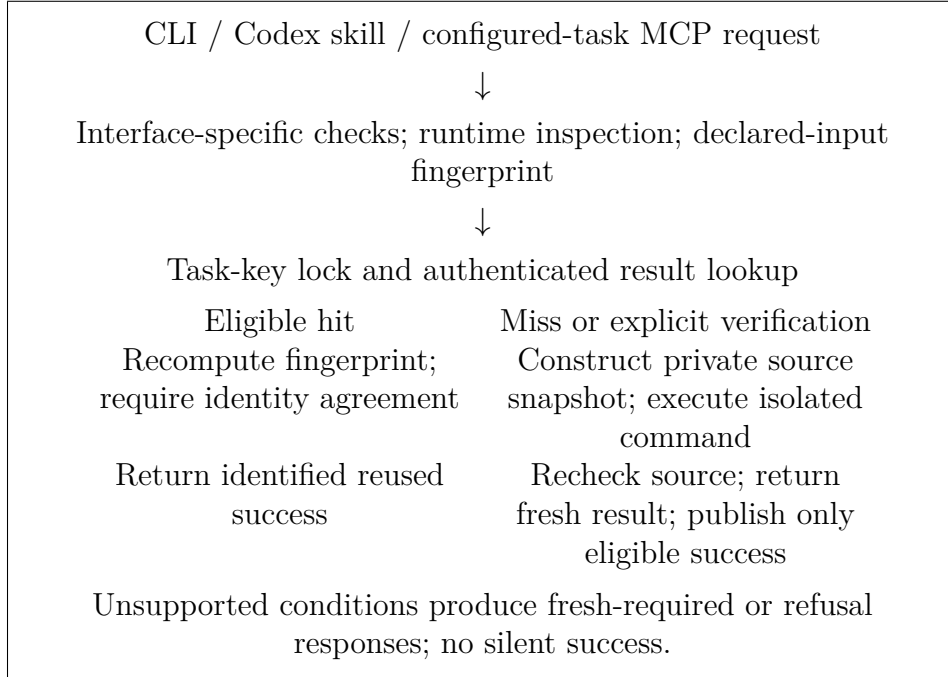


Figure 1: Whole-file result-reuse path. A hit returns status metadata, not the prior transcript. This explanatory diagram was revised as editable LaTeX with OpenAI Codex desktop 26.901.6511.0 assistance and checked against the described implementation; it is not experimental image data.

identity. A matching cache filename is insufficient: the entry must authenticate and satisfy the configured contract. On a miss, ZeroRun stages a private source copy and runs it read-only in a resource-constrained, network-disabled container. A post-execution check detects source drift before publishing a success. Failed commands do not become reusable successes. Explicit verification can quarantine a stored result that conflicts with fresh execution.

The optimized whole-file hit path computes the full fingerprint, locks and authenticates the entry, then independently recomputes the full fingerprint. It returns a hit only if both identities and their payloads agree. No repository code executes and no execution copy is constructed on this path. Misses, forced execution, verification, and symbol-projection tasks retain the existing snapshot path. The optimization changes the order of work, not the cache eligibility threshold.

This is conditional correctness, not a soundness proof for arbitrary Python. An undeclared dependency, nondeterministic test, or violated host assumption can invalidate reuse. Two filesystem scans are not an atomic snapshot against arbitrary concurrent host mutations. Unsupported or insufficiently reviewed work must execute fresh or remain refused.

2.2. Interfaces and functionality

The CLI exposes structured results identifying reused success, fresh success, fresh failure, uncertainty, and refusal. The Codex skill and repository-bound MCP server expose the same distinction. Normal agent-facing reuse additionally requires exact-configuration authorization outside repository-controlled files; generating observation candidates does not authorize reuse. Laboratory fixtures used by the experiments authenticate cache records but are not production operator reviews, and their private keys are excluded from the artifact.

The package also contains a fine-grained pytest path. Because omitted tests can affect fixtures, callbacks, shared state, or ordering, it conservatively requires fresh execution when qualification is incomplete. That path is included for reproducibility, but it is not the demonstrated acceleration mechanism in this article. Whole-task reuse does not require individual tests to be independent; it requires the complete requested command to satisfy the stated contract.

Hits do not reconstruct stdout or stderr. Clients requiring new diagnostics, coverage, or generated files must execute fresh. The public operating guide and manifest reference document installation, operator review, authorization, and this boundary; automated interface checks do not establish external adoption.

3. Illustrative examples

3.1. Repeat, failure, and restoration

The executable example starts with an empty store and a fixed pytest target. It executes a seed, reuses unchanged successes, executes an added failure twice, then finds the earlier success after byte restoration. The seven-response sequence is:

Request	Expected whole-task response
Seed	MISS_EXECUTED
Two unchanged repeats	HIT_REUSED each
Added failure; repeated failure	MISS_FAILED each
Restoration; restored repeat	HIT_REUSED each

The example records an independent fresh full-target check after every request, including hits. It is a constructed validation sequence, not an estimate of the repetition frequency in real development.

For a configured target named **tests**, the CLI form is:

```
zerorun --manifest .zerorun.json --json explain tests
zerorun --manifest .zerorun.json --json run tests
zerorun --manifest .zerorun.json --json run tests --verify
```

These commands inspect eligibility, request execution/reuse, and verify through fresh execution. The operator guide supplies the reviewed-manifest and authorization procedure; the commands do not establish input completeness. The quickstart creates a disposable target and retains response JSON.

3.2. Client decisions and integration evidence

The reference consumer checks response consistency, **status**, **exit_code**, and tool/protocol errors: **mode=reuse** alone is not success. It labels previous success explicitly, routes fresh failures to diagnostics, and requests fresh execution when required; it does not authorize tasks or execute its recommended fallback.

For the same configured task, MCP **run_tests** takes the following arguments for ordinary reuse eligibility and fresh verification, respectively:

```
{"task":"tests"}
{"task":"tests","verify":true}
```

The server is bound to the target repository and receives the operator’s external authority location. Requesting reuse does not imply that the response is a hit.

The following client-decision trials use one deterministic two-file synthetic fixture, separately from the issue-fix producer below. A scripted consumer passed 14 response cases; 11 installed-server checks covered eight discovery/diagnostic requests. Three earlier Codex 0.153.3 configurations remain unsuccessful overall: missing authority, client-side approval refusal, and an unsupported model-generated argument after a successful four-turn readiness/execution/reuse/verification core. Trial 3’s two separate fresh checks agreed, but its later failure prevented the remaining planned checks. The full trial ledger and transcripts are retained. A separately frozen API-guided demonstration supplied a neutral API card. After non-model readiness and seeding, the model selected reuse for a status-only need and fresh verification for a fresh-evidence need; both decisions and two separate fresh checks passed. Six fixed no-tool interpretation cases also passed, including failure, refusal and rejection of a hit when fresh diagnostics were required. All eight model turns are retained without retries. This is bounded documented use, not a causal documentation experiment. An account-free guide additionally reproduced refusal, authorized readiness, fresh success, reuse and verification through the actual server from a clean public clone and external installation with 36 matching runtime modules. The recorded installation took 18.6 s and the server check 11.3 s, excluding cloning, prior Docker setup and researcher preparation. These are researcher-executed checks, not independent user observations. These installation timings describe the historical 0.5.1 guide. Separate source-bound receipts verify the current 0.5.2 public installation and five-stage server workflow.

These synthetic trials used the CLI’s default model without an identified backing model. They describe the recorded client configuration, not a model-population accuracy estimate. The operating guide separates laboratory examples from operator-approved application use.

3.3. Issue-state validation handoffs

A separate, prospectively specified study uses public issue-state repositories with supplied regression tests. The frozen shortlist yielded 24 main cases across 8 repositories and 2 pilot cases across 1 repository; the original acquisition target was 30 main cases across 10 repositories. Selection used a fixed repository shortlist, metadata validity and hash ordering, not measured outcomes. Selected, unavailable and unexecuted cases remain in the ledger. In controlled reference-patch handoffs, both arms validate the same final source: a cold producer followed by a consumer, either fresh twice or ZeroRun seed

then reuse. Separate fresh oracles check the results; a diagnostics request is measured separately. These are imposed handoffs, not observed repetition frequencies.

Deployment	Cases	Blocks/hits	Chain (s)	Consumer (s)	With setup (s)
Copy pilot	2/2	4/4	42.79/60.89	5.17/2.08	55.31/73.75
Image recovered	2/2	4/4	63.35/79.11	7.94/2.35	64.53/80.11
Image repeat	2/2	4/4	73.76/81.67	8.82/2.53	76.76/85.10
Image main	2/24	4/4	61.57/47.27	7.63/1.11	61.90/47.54

Table 3: Fresh/ZeroRun paired costs, without pooling cohorts. Cases are completed/selected; chain and setup-inclusive times are summed across complete blocks, while consumer times are per-request means. Setup includes per-arm source/environment preparation, excluding common image construction and acquisition. Missing complete timings are dashes, not zero.

Copy pilot was 42.3% slower over the complete chain. Image recovered was 24.9% slower over the complete chain. Image repeat was 10.7% slower over the complete chain. Image main was 23.2% faster over the complete chain. The first image pilot resumed after an operator-reported VM pause and host-storage interruption. Its original records and the recovery amendment are retained; it is not certified uninterrupted timing.

The dependency-image preparation took 22.95 s, separately from chain ratios. It packages the frozen dependencies in an inspected container rather than copying them with each source snapshot. Image acquisition, preparation and independent-oracle work are not consumer latency. The recorded image is locally digest-bound; public pulling and an independent bit-identical recipe rebuild are not established.

The earlier failed image preparation took 24.40 s; its original sources, failure and costs are retained separately.

Acquisition and operator-review costs are outside paired totals; unmeasured preparation is not assigned zero. Fresh-oracle and diagnostic timings are retained separately in the reconciled evidence.

The separate coding-agent producer pilot requested gpt-6-astra: 1 model invocation among 2 selected cases, with 1 completed producer session. The public issue and regression tests were supplied, while the reference source solution was withheld. Captured transcripts, final source snapshots and unsuccessful states are retained. Another selected case stopped before model invocation because client-isolation preflight found an existing configuration. It remains in the selection denominator, not a failed model patch. Separate fresh execution verified 1 completed fix among 2 selected cases. A verified fix requires previously failing regression nodes to pass with the same collected

node set and a source patch within the recorded boundary. 1 final snapshot passed the selected target. The downstream scripted handoff completed 2 paired blocks; its timing excludes model production and does not measure an autonomous consumer decision. Summed fresh/ZeroRun costs were 31.39/39.89 s for the chain, 15.27/5.64 s for its consumers, and 31.86/41.43 s including per-arm setup, under the same exclusions as Table 3. The verified SQLGlot example concerns a MySQL `REPLACE INTO` parsing failure: the production patch preserves the statement as a command, and the original failing regression passes fresh. This verifies the supplied target, not full SQL semantic support. The image main is a budget-limited partial cohort: both complete cases are from pycparser, and unsupported or unexecuted selections do not contribute zero-cost observations.

3.4. Real AI-workflow observations

The read-only trajectory audit uses public SWE-rebench OpenHands records [10, 11]. After ten separate pilot rows, fixed SHA-256 ordering selected 128 of the remaining 67,064 rows without using repository, resolution, repetition, or performance. Raw responses, exclusions, the parser, and a rate-limited attempt followed by recovery of the same selection are retained.

Of the 128 selected episodes, 122 had the required action/observation pairing; six were excluded for unmatched execution records, not replaced. The valid set spans 104 repositories and 120 issues. It contains 745 supported direct pytest invocations and 120 exact-command repeat pairs. All repeats have intervening barriers: 112 contain a reported editor mutation and eight contain other unmeasured effects. These are observations about recorded actions, not 120 cache hits. The parser excludes compound-shell commands from the direct-command category and preserves clipped outputs and missing exit markers.

This example makes a practical distinction visible: repeated command text is an inadequate cache key for source-editing agents. Even an apparent restoration requires independent source and environment reconstruction. A purposefully selected django-envron episode provides a positive, bounded mechanism example. We reconstructed its recorded source edits, added a fresh check at the intermediate edited state, and ran four controlled requests (Table 4). After the inverse edit, the complete declared source/runtime identity and original target’s cache key rejoined; the restored request returned one explicitly identified reused success, agreeing with an independent fresh 14-test outcome. The intervening collection command selected no tests and exited 5; it remained a fresh failure, not a reusable success.

This standalone case ran after the timing VM interruption, with independently checked source bindings. Retained failed attempts exposed a missing

Controlled request	Actual response	Exit	Fresh nodes
Seed target	MISS_EXECUTED	0	14
Edited target (extra check)	MISS_EXECUTED	0	15
Restored collection	MISS_FAILED	5	0
Restored target	HIT_REUSED	0	14

Table 4: One source-restoration case. The 15-test request is additional counterfactual instrumentation, not a test call made by the recorded agent. Every row includes a separate fresh full-target oracle.

Git-fixture marker; a separately versioned producer initialized local meta-data while preserving the baseline’s masking guard. The successful run uses Python 3.12.14 and pytest 9.1.1, not the recorded Python 3.9 environment. It demonstrates this content-restoration mechanism, not historical output replay, unchanged autonomous-agent behavior, or population-wide benefit.

4. Impact

4.1. Reproducible validation and measured cost

Researchers can change a target or input boundary while preserving fresh-oracle comparisons and explicit denominators. The kit retains raw invocations, source hashes, fixed manifests, failures, setup receipts, and analysis scripts, making qualification behavior and unfavorable costs inspectable. Codex assisted software, experiment and analysis implementation and manuscript preparation. Separately implemented validators recompute outcomes from retained raw records; input-inventory and fresh-execution oracles do not substitute cached outcomes for their reference. These are automated cross-checks, not independent human replication.

The original comparison used five pinned Python libraries: packaging, pycparser, Colorama, Click, and more-itertools. Ordinary pytest is the uncached reference; disabling only the optimized hit path gives a snapshot-first ablation, not an independent competing cache. Two reversed-order seven-request sequences per library yielded 70 fresh-result agreements and 40 optimized-arm whole-task hits. The original mean conditional hit latency reduction against the ablation was 67.3–71.0%. Complete-sequence direct/optimized ratios were 0.85–2.09. For more-itertools, optimized execution was 16.6% slower overall than snapshot-first reuse, despite faster hits. These results are retained, including large lifecycle delays and their order sensitivity.

The prospectively recorded replication used six independently initialized blocks on each of the four previously observed short subjects, covering every ordering of direct pytest, snapshot-first reuse, and optimized reuse.

The seven request states, 900-second execution bound and container limits were unchanged. Timings cover complete helper/API invocations, including their different Docker lifecycle paths; host telemetry and receipt writing are outside every boundary. Preparation is separately retained. This is a replication on seen subjects, not a holdout or replacement for more-itertools. Across 24 completed planned blocks, all 168 requests agreed with their fresh full-target checks and showed the expected cache behavior, including 96 optimized hits and 72 non-hit requests. Table 5 separates full-sequence cost, setup, all-block spread and conditional hit savings. All 504 complete-block arm invocations and 168 separate fresh-oracle captures are retained. A VirtualBox host assertion interrupted execution after 21 complete blocks and part of Packaging block four. A recorded amendment reran only unfinished preselected blocks four through six with the unchanged driver. Original failures, partial records and recovery preflight failures remain available; this is recovered coverage, not uninterrupted completion. Packaging’s unflushed setup total is unavailable, so its setup-inclusive ratio is omitted. The interrupted attempt retains 1 complete request, 1 partial request and an outcomeless invocation. Including its surviving arm costs changes Packaging’s direct/optimized ratio to 1.029.

Subject	D/F	$D/F + \text{setup}$	Block range	Hit saving (%)
click	1.05	1.05	0.89–1.64	68.1
pycparser	0.89	0.91	0.78–1.04	68.8
colorama	1.01	1.01	0.98–1.11	67.6
packaging	1.04	—	0.88–1.32	70.8

Table 5: Complete replicated sequence cost versus conditional hit benefit. D/F is summed direct time divided by summed optimized time; above one favors reuse. Setup-inclusive ratios allocate common dependency preparation and each block’s materialization to both arms. Block range retains all six complete sequences. Hit saving compares median optimized and snapshot-first hit latency, not end-to-end agent time.

A post-hoc operating-region analysis retains every completed block and holds within-category composition fixed. Let D_h, F_h and D_m, F_m denote mean complete direct/optimized costs in hit and non-hit categories. For a hypothetical hit fraction p , fixed category means give the equality point

$$p^* = \frac{F_m - D_m}{(F_m - D_m) + (D_h - F_h)}.$$

The crossovers for Click, pycparser, Colorama and Packaging are 54.25%, 63.58%, 56.30%, 55.06%, respectively; larger fractions favor reuse in this

fixed-mixture calculation. The imposed fraction is 57.14%, explaining why full-sequence ratios can lie near or below one despite fast hits. All six block-level crossovers per subject are archived; their ranges are descriptive, not confidence intervals. The two fingerprints account for 73.0–75.9% of optimized hit time; their inclusive enclosing timer is not double-counted. This is not a measured agent hit frequency or a deployable admission policy. Different miss/failure mixtures and host costs change the threshold. Common setup cancels from equality; additional deployment/review costs remain unmeasured, not zero.

An independent standard-library input-inventory oracle checks boundary changes without importing ZeroRun’s fingerprint implementation. Linux validation covers 32 production/oracle comparisons and six explicit oracle-scope refusals. Cases include same-size content changes with restored modification time, helper/configuration/data edits, additions, deletion, renaming, directory modes, restoration, command and environment changes, missing inputs, and symlinks. A new *undeclared* top-level file changes neither declared identity: the test documents this limitation rather than claiming automatic closure discovery. These checks establish implementation behavior within the tested boundary, not universal determinism.

The engineering record retains historical source-bound Python 3.10–3.14 CI checks, Windows regressions and selected public-package tests, with separate counts and platform skips. A 100-repository CLI/skill/MCP exercise returned 99 passes and one safe refusal; it did not execute 100 upstream suites or autonomous tasks. A five-library revision evaluation recorded 200 fresh comparisons but no accepted fine-grained node reuse. Its product performance gates remain unmet. These evaluations are not pooled into a reliability percentage.

4.2. Scope, reuse, and limitations

Integrators can inspect the result contract and review boundary before enabling reuse; researchers can audit responses against implementation bytes, fresh outcomes, and full cost. The MIT release permits adaptation and commercial reuse of ZeroRun-owned code; third-party licenses remain separate. The evidence does not establish the prevalence of reusable tasks, population-wide speedup, model-token savings, improved patch quality, user productivity, or commercial demand. The trajectory audit is descriptive and resource-bounded; repository and issue clustering is reported. Its zero barrier-free pairs do not prove zero state-restoration opportunities, but they prevent interpreting repetition as measured eligibility. The controlled library targets are convenience workloads on one constrained VM, and original and replicated observations are not statistically independent datasets. The VM shared

a Windows host with publication and validation tooling; host activity outside the VM was not fully isolated. Complete-block spreads and influential observations are reported without treating thousands of nested test nodes as independent samples.

No external users, production deployments, or third-party publications using ZeroRun are claimed. Model sessions and controlled downstream handoffs are reported separately. The prevalence of reusable states, generalization across clients and models, and end-to-end workflow benefit remain unmeasured.

5. Conclusions

ZeroRun exposes identified test-status reuse to AI coding tools. Its whole-file hit path avoids staging while retaining identity checks and fresh execution. Runnable examples, fresh-oracle comparisons, boundary checks, and trajectory observations support conditional utility while exposing overhead and applicability limits. The release does not establish universal AI acceleration.

Data and software availability

Code, research data and reproduction instructions are deposited in the public ZeroRun repository [12]. Table 1 pins the submitted distribution. Its manifest separately retains the historical 0.5.1 study runtime at revision 86f42289c2f59a74b1f642f0b20d1a26b3d55e57; the 0.5.2 distribution changes version metadata and MCP discovery descriptions, with separate source and installation checks. Historical timings are not relabeled as a 0.5.2 replication. Supplementary material 1, `ZeroRun_SoftwareX_reviewer.zip`, contains inventoried software, raw evidence and analysis tools. Public trajectory responses retain CC-BY-4.0 notices and source attribution. Private cache-authentication keys and operator authority are excluded.

Funding

This research received no specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interests

Jishan Kapoor develops ZeroRun and has an interest in its potential commercialization. No external funding or established commercial adoption is reported.

Declaration of generative AI and AI-assisted technologies

OpenAI Codex was used extensively to assist software and experiment implementation, analysis, literature checking, manuscript drafting, and document preparation. It is not an author. The sole human author is responsible for reviewing the evidence, verifying references and claims, and approving the submitted version.

References

- [1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan, SWE-bench: Can language models resolve real-world GitHub issues?, in: The Twelfth International Conference on Learning Representations, 2024.
URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html
- [2] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, O. Press, SWE-agent: Agent-computer interfaces enable automated software engineering, in: Advances in Neural Information Processing Systems 37, 2024. doi:10.52202/079017-1601.
- [3] G. Rothermel, M. J. Harrold, A safe, efficient regression test selection technique, ACM Transactions on Software Engineering and Methodology 6 (2) (1997) 173–210. doi:10.1145/248233.248262.
- [4] S. Yoo, M. Harman, Regression testing minimization, selection and prioritization: A survey, Software Testing, Verification and Reliability 22 (2) (2012) 67–120. doi:10.1002/stvr.430.
- [5] J. F. Aldana-Martín, A. Benítez-Hidalgo, J. F. Aldana-Montes, eidos: A modular approach to external function integration in LLMs, SoftwareX 31 (2025) 102290. doi:10.1016/j.softx.2025.102290.
URL <https://doi.org/10.1016/j.softx.2025.102290>
- [6] M. Couto, J. Parapar, D. E. Losada, LabChain: Enabling reproducible and modular scientific experiments in Python, SoftwareX 33 (2026) 102543. doi:10.1016/j.softx.2026.102543.
URL <https://doi.org/10.1016/j.softx.2026.102543>
- [7] Bazel Project, Remote caching, official documentation; accessed 2026-09-05 (2026).
URL <https://bazel.build/remote/caching>
- [8] Y. Zhai, D. Shen, J. Luo, B. Yang, ToolCaching: Towards efficient caching for LLM tool-calling, preprint, version 1 (2026). arXiv:2601.15335, doi:10.48550/arXiv.2601.15335.
URL <https://arxiv.org/abs/2601.15335>

- [9] A. V. Kumar, B. Kataria, B. Oh, E. Manzoor, R. Singh, TVCache: A stateful tool-value cache for post-training LLM agents, preprint, version 1 (2026). [arXiv:2602.10986](#), doi:10.48550/arXiv.2602.10986.
URL <https://arxiv.org/abs/2602.10986>
- [10] M. Trofimova, A. Shevtsov, I. Badertdinov, K. Pyaev, S. Karasik, A. Golubev, OpenHands trajectories with Qwen3-Coder-480B-A35B-Instruct, nebius dataset and accompanying blog, CC-BY-4.0; authors and year from the archived dataset citation. Observed revision 35455389ab51bf5e2306bfd436ef72d0f98bf882; accessed 6 September 2026. Exact retrieved responses archived with the article (2025).
URL <https://huggingface.co/datasets/nebius/SWE-rebench-openhands-trajectories>
- [11] I. Badertdinov, A. Golubev, M. Nekrashevich, A. Shevtsov, S. Karasik, A. Andriushchenko, M. Trofimova, D. Litvintseva, B. Yangel, SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents, in: Advances in Neural Information Processing Systems 38, 2025. doi:10.52202/085713-0788.
URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/21bec6ace947b1b58967b945c8ac0f10-Abstract-Datasets_and_Benchmarks_Track.html
- [12] J. Kapoor, ZeroRun: Research software and source-bound evaluation data, [dataset and software] GitHub, distribution version 0.5.2 with historical 0.5.1 study runtime retained; immutable source/evidence revision identified in the article’s code metadata. Accessed 7 September 2026 (2026).
URL <https://github.com/floxy-21/zerorun-research>

Current executable software version

ZeroRun 0.5.2, Python command-line package. The public release includes installation instructions, automated checks, and the separately identified historical 0.5.1 study runtime. Linux/amd64 Docker execution is required for the demonstrated whole-task reuse mode.